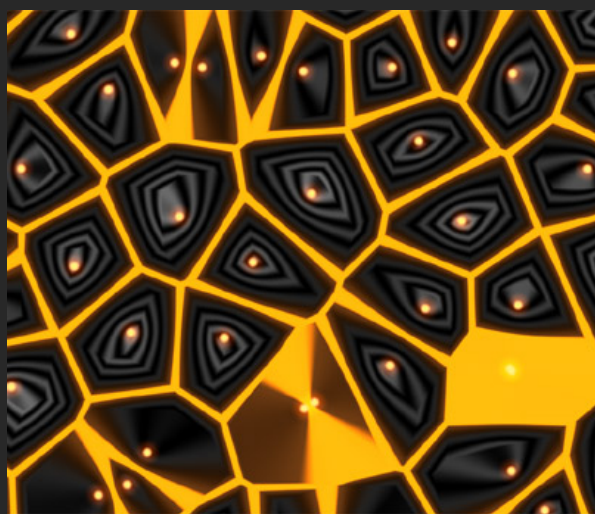


Inigo Quilez :: articles :: voronoi edges - 2012



Intro

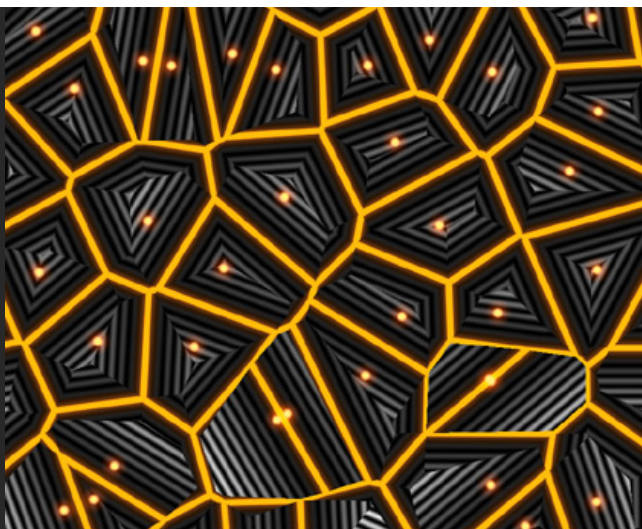
Anyone who has used voronoi procedurals to create a reptile skin texture or dry ground tiles know that when taking $F2 - F1$ and thresholding it, the width of the lines separating the cells have inconsistent width. Which is an annoying problem. Techniques exist to alleviate this problem, but they are only approximations. Here we will find the implementation of the voronoi noise that generates mathematically perfect cell separation lines, and which is full procedural (and runs at a few hundred frames per second in my laptop's GPU).



Simple $F2-F1$ voronoi leads to non uniform lines, corrupted cells and broken cell isolines



The new algorithm provides the correct distance metric. Lines are correct, even at the connection points, and isolines equally spaced.



The wrong approximate solution. Note the funny line widths and the broken cell isolines

The problem

The problem is that subtracting the distance to the closest point from the distance to the second closest point, or what people know as *F2-F1 Voronoi*, it's pretty close to a cell border generator. Indeed, the borders of the cells happen at locations where these two distances are equal (equidistant points from the two closest neighbors), so the function $F2-F1$ takes value 0.0 exactly at the borders of the cells, which is super usefull. So, i'd say it's pretty normal that one is tempted to simply smoothstep $F2-F1$ between two small numbers and call that a "cell edge". That sort of works, but not completelly. $F2-F1$ is not a distance really, as it expands and contracts depending on the distance between the the two cell points in each side of the edge, which can change wildly over the domain of the voronoi. Anyway, for reference, the implementation would be like this:

```
vec2 voronoi( in vec2 x )
{
    ivec2 p = floor( x );
    vec2 f = fract( x );

    vec2 res = vec2( 8.0 );
    for( int j=-1; j<=1; j++ )
        for( int i=-1; i<=1; i++ )
        {
            ivec2 b = ivec2(i, j);
            vec2 r = vec2(b) - f + random2f(p + b);
```

```
float d = dot(r, r);

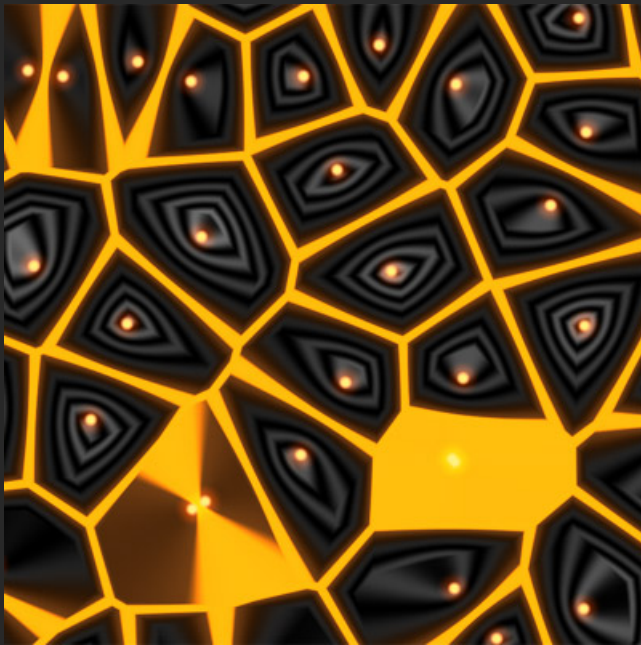
if( d < res.x )
{
    res.y = res.x;
    res.x = d;
}
else if( d < res.y )
{
    res.y = d;
}

return sqrt( res );
}

float getBorder( in vec2 p )
{
    vec2 c = voronoi( p );

    float dis = c.y - c.x;

    return 1.0 - smoothstep(0.0,0.05,dis);
}
```



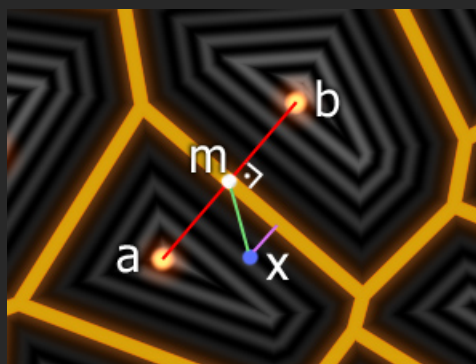
As it can be seen in the image to the right, and in the header of the article, this naive thresholding doesn't give good results. Some borders are thin, and others are so thick they completely take over the cell and make it

black (or yellow in the picture in the header).

Some ways to *almost* solve the problem

One simple approach to estimate the real distance would be to take the function $F_2 - F_1$, and evaluate its gradient, then divide $F_2 - F_1$ by the modulo of the gradient. You can learn how to do that in [this article](#). This is a generic method in fact, and it works okeish in most cases (but not always). The problem is, of course, it's very slow cause it needs three extra voronoi evaluations. Thing is we can do much better than that.

Good voronoi implementations not only return the distance to the closest points, but also the points themselves (both position and ID). This is useful for a number of reasons involving coordinate systems for interesting shading, etc. Not important right now. But, if we know where the closest two points of our voronoi grid are, then we can probably approximate much better the distance to the line that separates the cells.



Of course, that line is the line that bisects the red segment between these two points a and b . It passes through the point m which is just the average of a and b . The direction vector of the border (yellow) is perpendicular to the direction of the red segment, and can of course be obtained by swapping and negating one of the coordinates of the $b - a$ red segment direction. The blue point x is the point we are shading, and the distance to the border is the length of the purple segment. That's what we are after. So, we just have to project the vector $x - m$ along a normalized version of $b - a$, which gives us the length of the purple vector.

$$distance = \left\langle x - \frac{a+b}{2}, \frac{b-a}{|b-a|} \right\rangle$$

The very cool thing of not working in "world/object" space is that we already have the coordinates of a and b relative to x in the implementation of the voronoi itself, hence we can evaluate our distance in a coordinate frame centered at x , and remove x from the equation. This makes the code smaller, and less sensitive to precision issues.

The code for this attempt to fixing the problem would then be:

```
vec2 voronoi( in vec2 x, out vec2 oA, out vec2 oB )
{
    ivec2 p = floor( x );
    vec2 f = fract( x );

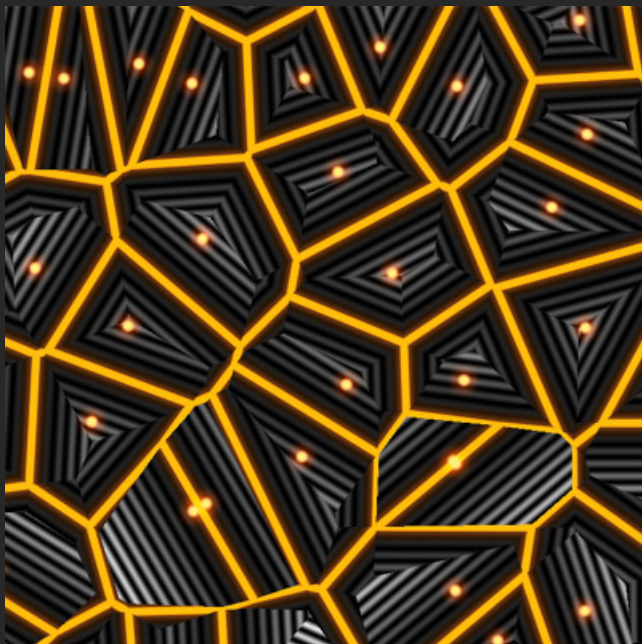
    vec2 res = vec2( 8.0 );
    for( int j=-1; j<=1; j++ )
    for( int i=-1; i<=1; i++ )
    {
        ivec2 b = ivec2(i, j);
        vec2 r = vec2(b) - f + random2f(p+b);
        float d = dot( r, r );

        if( d < res.x )
        {
            res.y = res.x;
            res.x = d;
            oA = r;
        }
        else if( d < res.y )
        {
            res.y = d;
            oB = r;
        }
    }

    return sqrt( res );
}

float getBorder( in vec2 p )
{
    vec2 a, b;
```

```
vec2 c = voronoi( p, a, b );  
  
float d = dot(0.5*(a+b),normalize(b-a));  
  
return 1.0 - smoothstep(0.0,0.05,d);  
}
```



The image resulting from changing these three lines of code is much better - there's no broken black cells, and the lines are all the same size. More or less. Because the image is still not perfect just yet, the thickness of the lines is still doing some funny things. This is because in the inside of the cells the second closest point does still change discontinuously, meaning that sometimes, especially near the corners where three lines meet, we are computing the distance to the wrong border line. The error manifests itself pretty well in the cell distance isolines, which actually make no sense for many of the cells.

The final algorithm

Ok then, the solution must be to first detect which cell contains the closest point to our shading point x , then do the neighbor search centered at that cell instead of the cell that contains x . One way to do that is implementing this, indeed, in two passes. That will make things slower of course, but it will still be a constant time algorithm (in fact, the code runs above 100 frames per second - 10 milisecond - in my laptop at 1280x720

resolution). Note that in the second loop we are relying on *normalize()* to create an *inf* in case *r* equals *mr*.

```
float voronoiDistance( in vec2 x )
{
    ivec2 p = ivec2(floor( x ));
    vec2 f = fract( x );

    ivec2 mb;
    vec2 mr;

    float res = 8.0;
    for( int j=-1; j<=1; j++ )
    for( int i=-1; i<=1; i++ )
    {
        ivec2 b = ivec2(i, j);
        vec2 r = vec2(b) + random2f(p+b)-f;
        float d = dot(r,r);

        if( d < res )
        {
            res = d;
            mr = r;
            mb = b;
        }
    }

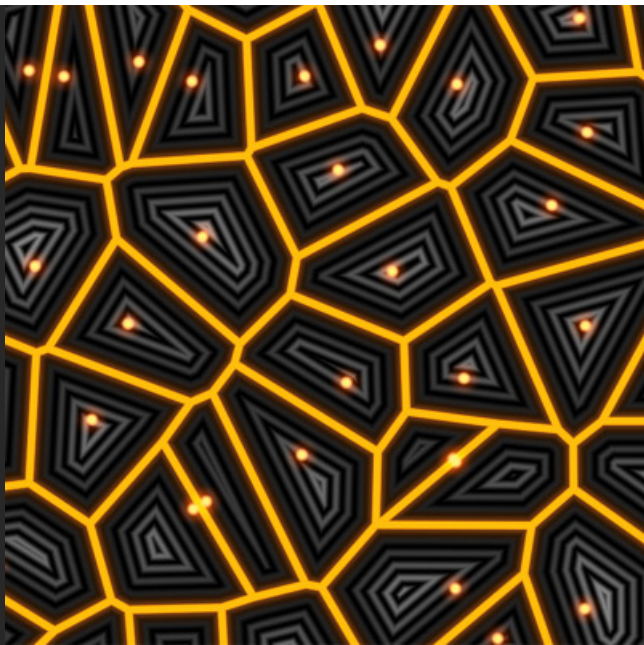
    res = 8.0;
    for( int j=-2; j<=2; j++ )
    for( int i=-2; i<=2; i++ )
    {
        ivec2 b = mb + ivec2(i, j);
        vec2 r = vec2(b) + random2f(p+b) - f;
        float d = dot(0.5*(mr+r), normalize(r-mr));

        res = min( res, d );
    }

    return res;
}

float getBorder( in vec2 p )
{
    float d = voronoiDistance( p );

    return 1.0 - smoothstep(0.0,0.05,d);
}
```



One interesting thing to do would be to implement this in a single pass. The other thing pending for me to do is to identify the equidistant point within the each cell, which does NOT coincide with the jittered point that originated the cell. That's why in the pictures above the glowing dot (the originating point of the cell) does not lay in the place where all the isolines collapse.

inigo quilez - learning computer graphics since 1994